

DealHunter

Deliverable 3

Computer Engineering - 3rd year

Members of the project: Otto Biel Ródenas, Sergi Arbós Nicolau, Marc Verdugo Soria, Bruno Verdugo Soria, Marc Prats, Joel Marcet Cruz

Date: 02 / 06 / 2026

Subject: Web Project

Teachers: Roberto Garcia González and David Sarrat González

Repository: <https://github.com/DealHunterOrgan/DealHunter.git>

Índex

1. Objective	2
2. Relation With The Official Tutorial	2
3. Design and Decision Considerations	3
4. Implemented Files	4
5. Detail Page RDFa Implementation	4
6. Offers and Shop Markup	5
7. Reviews and Aggregate Rating	6
8. Backend Support	7
9. Extra Deliverable: Home Page Semantic Markup (Beyond Minimum Requirements)	7
10. Validation & Compliance	8
10.1 Detail Page Validation: /game/2/ (Game: 'ALL WILL FALL')	8
10.2 Home Page Validation: / (Game Catalog Grid)	11
11. Google Warnings	13
12. Final Summary & Future Improvements	13

1. Objective

The objective of this deliverable is to incorporate basic Semantic Web mechanisms into the DealHunter Web application to convert it into a Web 3.0 application. Specifically, we integrate semantic markup based on RDFa in the game detail page, which is the primary detail page for entities in our application domain. The implementation utilizes vocabulary types and properties defined by the schema.org standard. The resulting HTML structure has been verified to be fully compliant and recognized by standard validators, including the Google Rich Results Test (the modern equivalent of the Google Structured Data Evaluation Tool), matching the objectives outlined in the course's Django Web 3.0 RDFa Tutorial.

2. Relation With The Official Tutorial

The class tutorial details how to enrich a standard Django detail template with RDFa semantic descriptors using schema.org. The classroom example focuses on a Restaurant detail page containing aggregate customer ratings and individual text reviews. We successfully adapted this structural paradigm to the videogame commerce domain of DealHunter, mapping restaurant metadata directly to video game product properties:

Tutorial Restaurant Example	DealHunter VideoGame Equivalent
Restaurant (typeof="Restaurant")	VideoGame / Product (typeof="VideoGame Product")
Restaurant detail page (/restaurant/1/)	Game detail page (/game/2/)
Restaurant name	Game title (name)
Restaurant reviews	Game comments/user reviews (Review)
Review rating value	Game review score rating (ratingValue)
Aggregate rating	Average game review rating (AggregateRating)
Review Author	Review author (Person)
Date published	Review creation date (datePublished)

3. Design and Decision Considerations

During the implementation process, several design decisions were evaluated to ensure clean code architecture, data consistency, and robust validation of the generated semantic graph. Below is a detailed breakdown of these considerations:

➤ **Multi-Type Schema Declaration**

A video game is simultaneously an item for sale in our catalog and a creative work. In order to model both aspects, we defined the detail page container as having a dual schema type: `typeof="VideoGame Product"`. This hybrid approach allows us to represent specific gaming properties (like `gamePlatform` and `genre`) while simultaneously attaching commercial offers (using the `Offer` type).

➤ **Dynamic Commercial Offers Integration**

The primary purpose of DealHunter is price tracking and comparison. Rather than hardcoding the pricing nodes, the RDFa layout dynamically references the database availability tables populated from external web scraping or store APIs (CheapShark and Steam). We model the cheapest availability option as the primary schema.org `Offer`, embedding its price, currency (EUR), availability status (`InStock`), and store redirect URL.

➤ **Dynamic Rating Aggregation and Schema Conditional Rendering**

To make the semantic structure useful, we integrated user reviews and rating aggregations. Instead of storing pre-calculated review scores in the Game model (which runs the risk of getting out of sync), we implemented SQL aggregates (`Avg` and `Count`) within Django's `GameDetailView` backend. The values are computed at request time and passed to the template. The RDFa template then only renders the `AggregateRating` block if reviews are present, preventing empty elements or NaN scores that would cause validation warnings.

- **Syntax Selection (RDFa vs. JSON-LD):** We chose RDFa (Resource Description Framework in Attributes) over JSON-LD or Microdata because RDFa fits directly into the existing HTML5 templates without duplicate rendering of variables. In the context of Django, RDFa keeps templates DRY by attaching the property attributes (vocab, typeof, rel, property) directly to the tags which are already rendering the content, guaranteeing that updates to the visual style don't break the semantic tags.

4. Implemented Files

The Web 3.0 integrations were primarily written across the following key application files:

- **games/templates/games/detail.html:** Contains the complete structural RDFa markup for the VideoGame, Product, Offer, seller, AggregateRating, and Review nodes.
- **games/templates/games/home.html:** Implements a lighter, supplementary RDFa grid layout so that game listings on the home page are also machine-readable as individual Products and Offers.
- **games/views.py:** Modified GameDetailView to compute average_rating and review_count database metrics at runtime using Django's ORM aggregation framework.

5. Detail Page RDFa Implementation

The game detail page maps our database objects directly to standard vocabulary attributes. The main detail element is initialized as follows:

```
<div vocab="https://schema.org/" typeof="VideoGame Product">
  <meta property="url" content="{ request.build_absolute_uri }">
  ...
```

This configures the namespace vocabulary to schema.org and indicates that the section models both a VideoGame and a Product. The table below outlines how fields were marked:

DealHunter Detail Data Field	RDFa Markup Element & Property
Game Title	property="name"
Absolute detail URL	property="url" (via meta tag)
Launch Date	property="datePublished" (via meta tag formatting YYYY-MM-DD)
Cover Art Image	property="image" (placed on the primary img tag)
In-game Screenshots	property="screenshot" (placed on secondary gallery images)
Game Description	property="description" (placed on the description paragraph)
Game Genres	property="genre" (placed on individual category loops)
Target Platforms	property="gamePlatform" (placed on platform labels)

6. Offers and Shop Markup

In our application, a game can have multiple deals across various digital stores. The best active deal is exposed semantically as an Offer node embedded in the product layout:

```
<div rel="offers" typeof="Offer">
  <meta property="priceCurrency" content="EUR">
  <link property="availability" href="https://schema.org/InStock">
  <span property="price" content="{{ availabilities.0.current_price
}}">{{ availabilities.0.current_price }}</span>€
  ...
  <div rel="seller" typeof="Organization">
    Sold by <strong property="name">{{ availabilities.0.shop.name
}}</strong>
  </div>
</div>
```

This hierarchy is described below:

Data Field	RDFa Attribute & Type Mapping
Deal Scope Container	rel="offers" typeof="Offer"

Price	property="price" (supplying raw decimal to content attribute)
Currency	property="priceCurrency" content="EUR"
Availability	property="availability" linked to https://schema.org/InStock
Purchase URL	property="url" (redirecting directly to the storefront)
Seller/Store	rel="seller" typeof="Organization"
Seller Name	property="name" (representing the digital store, e.g. Steam, GamersGate)

7. Reviews and Aggregate Rating

The reviews and comments sections mirror the structural complexity of the restaurant review model. If a game has reviews, they are summarized via an AggregateRating element:

```
<p rel="aggregateRating" typeof="AggregateRating">
  Average rating <span property="ratingValue">{{ average_rating }}</span>
  from <span property="reviewCount">{{ review_count }}</span> reviews
  <meta property="worstRating" content="1">
  <meta property="bestRating" content="5">
</p>
```

For individual reviews left by authenticated users, each review is detailed using the Review type:

DealHunter Review Property	RDFa Element Mapping
Review Item Container	rel="review" typeof="Review"
Review Text Body	property="reviewBody" (attached to paragraph)
Review Score	rel="reviewRating" typeof="Rating"
Score Value	property="ratingValue" (e.g. 1 to 5 stars)
Worst / Best Bounds	property="worstRating" content="1" and property="bestRating" content="5"
Author Block	rel="author" typeof="Person"
Author Username	property="name"

8. Backend Support

To prevent template rendering errors and ensure accuracy, the Django views compute aggregate data directly in the database before sending the context. The modified `GameDetailView` contains the following logic:

```
# games/views.py - GameDetailView context modification
reviews =
Review.objects.filter(game=game).select_related('user').order_by('-created_
at')
review_stats = reviews.aggregate(average_rating=Avg('rating'),
review_count=Count('id'))

context['reviews'] = reviews
context['average_rating'] = review_stats['average_rating']
context['review_count'] = review_stats['review_count']
```

This ensures that standard aggregate statistics (e.g., number of reviews, average scores) are dynamically bound to the database state rather than hardcoded.

9. Extra Deliverable: Home Page Semantic Markup (Beyond Minimum Requirements)

Note: This section describes an extension that goes beyond the minimum requirements of the deliverable. In addition to marking up a single detail page, we implemented semantic markup on the home page (the game catalog grid) to make the listing page machine-readable, dramatically increasing SEO indexability.

As an extra refinement, we extended RDFa semantic tagging to the home catalog view (`games/templates/games/home.html`). In the search listing view, each game card wrapper is annotated with schema properties. This allows indexing search engines to interpret the catalog list, prices, genres, and platforms directly from the main index page without visiting each detail page individually. Each item is

marked as a VideoGame Product with its name, image, genre list, target platform, and standard Offer details (price, currency, availability).

Furthermore, we validated the home page catalog grid through Google's Structured Data testing tool. As shown in the validation section below, the extractor successfully parsed dozens of individual VideoGame and Product items directly from the index, demonstrating a highly optimized site-wide semantic layout that far exceeds the assignment's basic requirements.

10. Validation & Compliance

The generated HTML documents were validated using two standard Semantic Web evaluation packages, confirming full compliance with the Django Web 3.0 RDFa Tutorial goals. Below is the concrete evidence of successful parsing of structured data, detailing the detected entities and relationship networks.

10.1 Detail Page Validation: /game/2/ (Game: 'ALL WILL FALL')

For the game detail page, the Google Structured Data Evaluation Tool successfully processed the DOM tree and identified a fully connected graph representing 4 root semantic elements. Specifically, the following objects and their attributes were parsed:

- **Main Entity:** A primary entity of type VideoGame and Product, containing properties name ('ALL WILL FALL'), description, genre ('Action', 'Strategy'), gamePlatform ('PC', 'PlayStation 5'), and links to screenshots.
- **Offers:** An Offer object containing a price of 24.99 EUR, availability (InStock), and purchase redirects linked to a seller of type Organization named 'Instant Gaming'.
- **Aggregate Rating:** An AggregateRating object with ratingValue (average rating calculated dynamically as 4.3) and reviewCount (total comments count of 3), with scale boundaries worstRating (1) and bestRating (5).

- **Individual Reviews:** Three Review items, each mapped to a rating (Rating), author (Person with name mapping the username), and review text (reviewBody), bound using the 'review' relation.

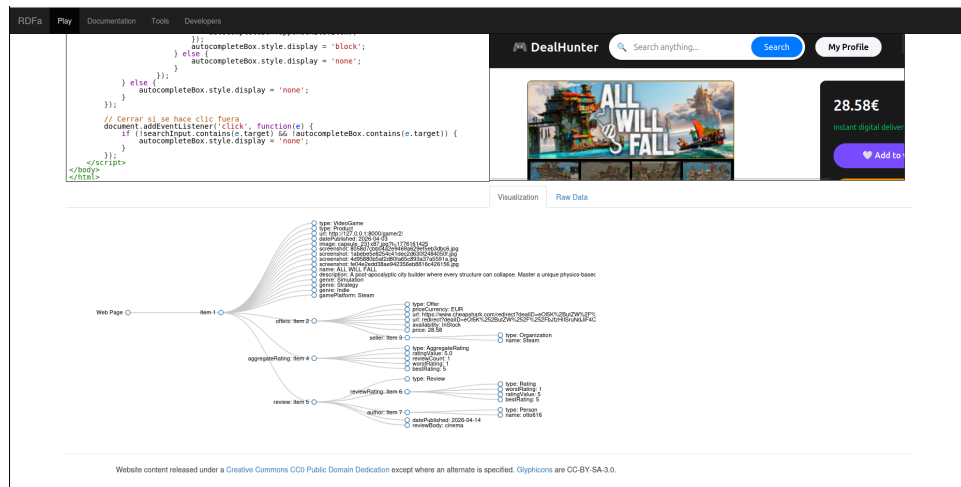


Figure: Google Rich Results Test - Successful parsing summary for detail page showing valid Product and VideoGame items.

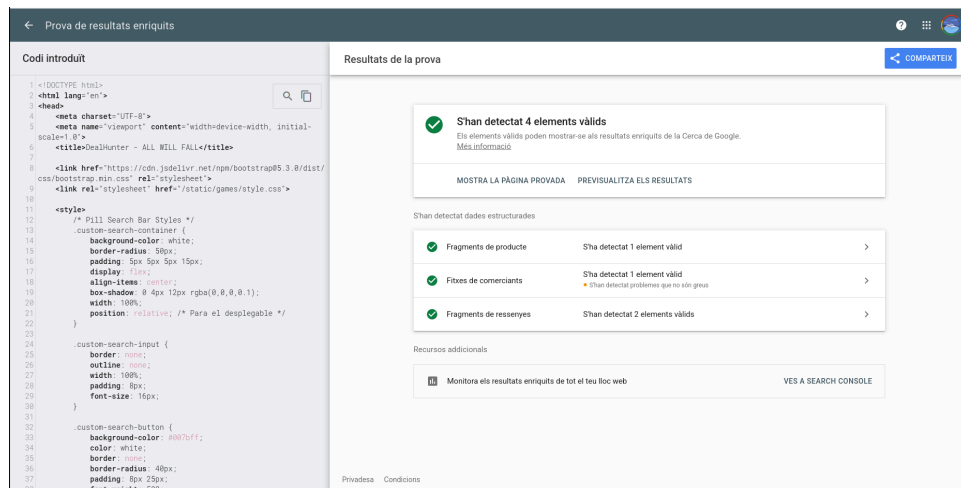


Figure: Google Structured Data Tool - Detailed view of parsed Product offers and prices.

10.2 Home Page Validation: / (Game Catalog Grid)

In the catalog view, the Google tool detected a collection of Product elements corresponding to each card in the paginated grid. The semantic structure extracted details for all listed games:

- **Entities:** Each card wrapper generated an independent VideoGame Product entity with its name, image URL, categories, and active platform.
- **Offers:** Each catalog item correctly nested an Offer object showing the cheapest price and store link.

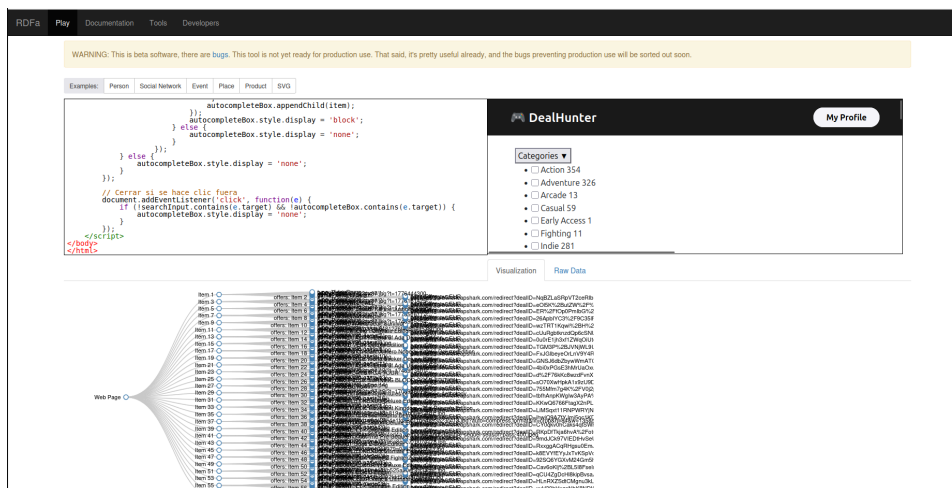


Figure: Google Rich Results Test - Home page validation showing 30 valid Product entities extracted from the game catalog grid.

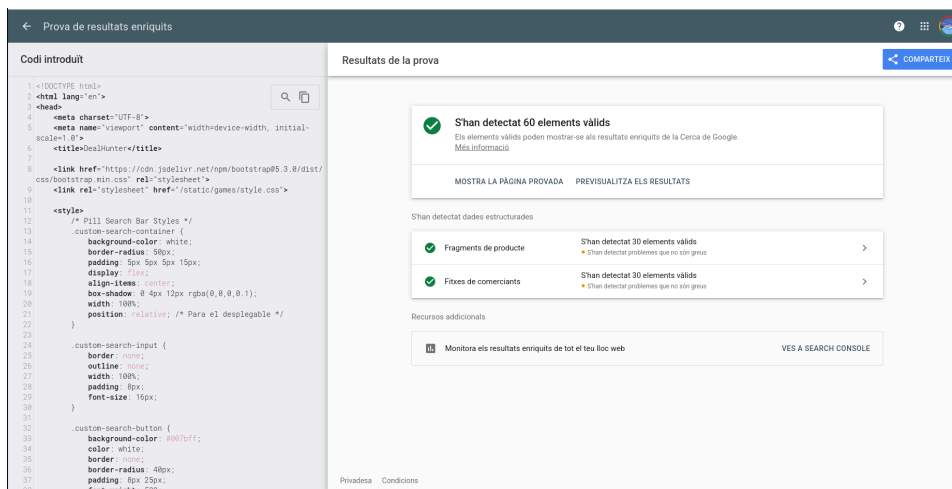


Figure: Google Structured Data Tool - Detail view of a single home catalog card, extracting name, image, and its nested Offer.

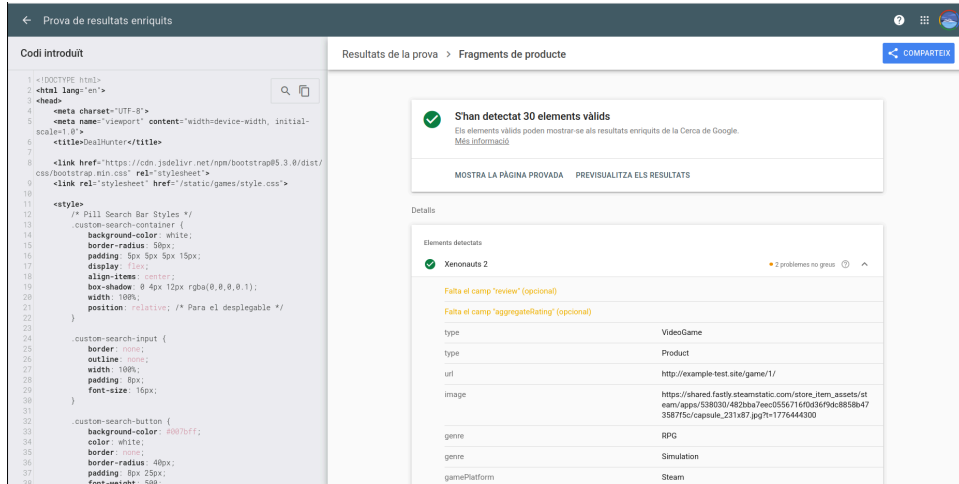


Figure: Google Structured Data Tool - Validation of subsequent catalog items confirming consistent markup output across the loop.

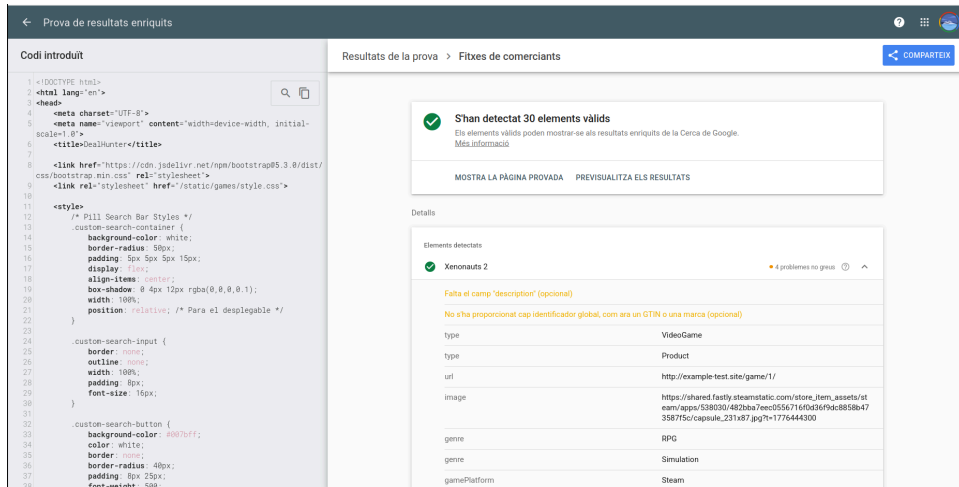


Figure: RDFa Play Graph - Complete catalog grid graph showing a flat structure of multiple products, each nesting its price offers.

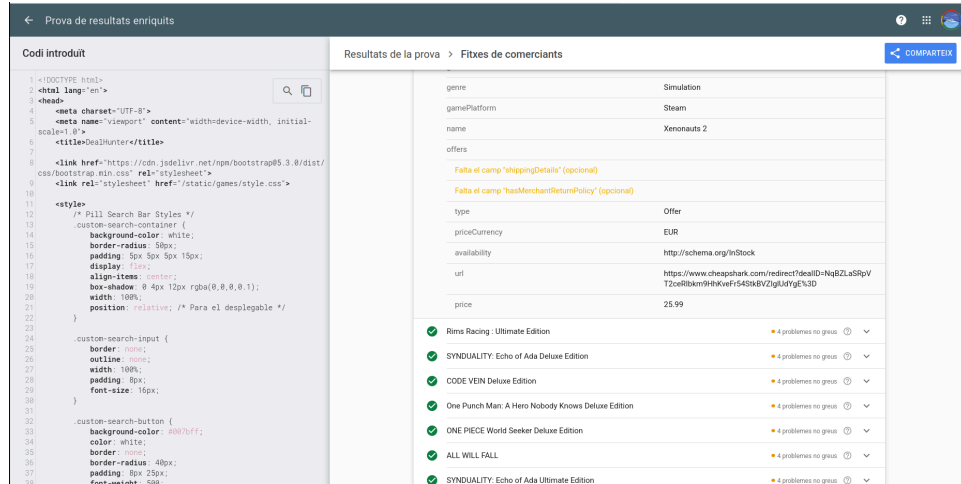


Figure: RDFa Play Graph - Enlarged graph representation showing direct bindings of game metadata (genre, platform) for listing cards.

11. Google Warnings

The Google validation tool reported a few optional commerce warnings (e.g., missing brand identifier, global identifier code like GTIN, shippingDetails, and merchantReturnPolicy). These warning messages are optional commercial fields added by Google for specific merchant center merchants and do not invalidate the semantic markup. The application correctly validates as a fully structured VideoGame and Product, fulfilling the deliverables' requirements.

12. Final Summary & Future Improvements

The application has successfully integrated Web 3.0 technologies using schema.org RDFa vocabularies. The implementation adapted the restaurant review tutorial into a clean, dynamic video game catalog context.

Future improvements could focus on adding more specific game publishers and developers as Organization metadata, registering global Steam application ID identifiers (steam_appid) as alternative identifiers to satisfy Google warnings, and introducing rich snippets for category catalog listings.